Agenda

- Association
- Inheritance
- super keyword
- Method Overriding
- ~~Upcasting & Downcasting~~
- ~~Final Keyword~~
- ~~Object class~~
- ~~toString()~~
- ~~equals()~~

Association

- If "has-a" relationship exist between the types, then use association.
- To implement association, we should declare instance/collection of inner class as a field inside another class.
- There are two types of associations
    - 1. Composition
    - 2. Aggregation

1. Aggegration

- Represents has-a relation i.e. loose coupling between the objects.
- The inner object can be added, removed, or replaced easily in outer object.
- Car has a Driver.
- Company has Employees.
- Room has a window
- Employee has a vehicle

2. Composition

- Represents part-of relation i.e. tight coupling between the objects.
- The inner object is essential part of outer object.
- Heart is part of Human.
- Engine is part of Car.
- Wall is part of Room.
- joining date is a part of employee

```
public class Date {
private int day;
private int month;
private int year;

public Date() {
}

public Date(int day, int month, int year) {
```

```
    this.day = day;
    this.month = month;
    this.year = year;
  }

  public void acceptDate() {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter day = ");
    day = sc.nextInt();
    System.out.print("Enter month = ");
    month = sc.nextInt();
    System.out.print("Enter year = ");
    year = sc.nextInt();
  }

  public void displayDate() {
    System.out.println(day+"/"+month+"/"+year);
  }
  }

  public class Customer{
    private String name;
    private String mobile;
    private Date dob; // Aggegration

  public Customer() {
    this.name = "";
    this.mobile = "";
    this.dob = null;
  }

  public void acceptCustomer() {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter name = ");
    name = sc.next();

    System.out.print("Enter mobile = ");
    mobile = sc.next();
  }

  public void displayCustomer() {
    System.out.println("Name = " + name);
    System.out.println("Mobile = " + mobile);
    if (dob != null) {
        System.out.print("Date of Birth = ");
      dob.displayDate();
    }
  }

  // Aggregation
  public void setDob(Date dob) {
    this.dob = dob;
  }
  }
```

```
public class Employee {
    private int id;
    private String name;
    private double salary;
    private final Date joining;

    public Employee() {
        this.id = 0;
        this.name = "";
        this.salary = 0.0;
        this.joining = new Date(); // Composition
    }

    public void acceptEmployee() {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter id = ");
        id = sc.nextInt();

        System.out.print("Enter name = ");
        name = sc.next();

        System.out.print("Enter salary = ");
        salary = sc.nextDouble();

        this.joining.acceptDate();
    }

    public void displayEmployee() {
        System.out.println("id = " + id);
        System.out.println("Name = " + name);
        System.out.println("Salary = " + salary);
        System.out.print("Date of Joining = ");
        this.joining.displayDate();
    }
}
```


Inheritance

- If "is-a"/"kind-of" relationship exist between the types, then use inheritance.
- Inheritance is process of generalization to specialization.
- All members of parent class are inherited to the child class.
- Parent class is also called as super class and child class is also called as sub-class.
- Example:
    - Manager is a Employee
    - Mango is a Fruit
    - Rectangle is a Shape
- In Java, inheritance is done using extends keyword.
- Java doesn't support multiple implementation inheritance i.e. a class cannot be inherited from multiple super-classes.

- However Java does support multiple interface inheritance i.e. a class can be inherited from multiple super interfaces.

```
class Employee {
private int id;
private String name;
private double salary;

public Employee() {
    System.out.println("Employee Ctor");
}

public void acceptEmployee() {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter id = ");
    id = sc.nextInt();

    System.out.print("Enter name = ");
    name = sc.next();

    System.out.print("Enter salary = ");
    salary = sc.nextDouble();

}

public void displayEmployee() {
    System.out.println("id = " + id);
    System.out.println("Name = " + name);
    System.out.println("Salary = " + salary);
}
}

class Manager extends Employee {
    private double bonus;

    public Manager() {
        System.out.println("Manager Ctor");
    }

    public void acceptManager() {
        Scanner sc = new Scanner(System.in);

        this.acceptEmployee();

        System.out.print("Enter bonus = ");
        bonus = sc.nextDouble();

    }

    public void displayManager() {
        displayEmployee();
        System.out.println("Bonus = " + bonus);
```

```
    }

}

class Salesman extends Employee {
    private int salesCount;
    private double commission;

    public Salesman() {
        System.out.println("Salesman Ctor");
    }

    public void acceptSalesman() {
        Scanner sc = new Scanner(System.in);

        this.acceptEmployee();

        System.out.print("Enter sales count = ");
        salesCount = sc.nextInt();
        System.out.print("Enter commission = ");
        commission = sc.nextDouble();

    }

    public void displaySalesman() {
        displayEmployee();
        System.out.println("SalesCount = " + salesCount);
        System.out.println("Commission = " + commission);
    }

}
```

Types of Inheritance

- 1. Single

```
class A {

}
class B extends A{

}
```

- 2. Multiple

```
class A {

}
class B {
```

```
}
class C extends A,B{ // Not Allowed

}

interface I1{

}
interface I2{

}

interface I3 extends I1,I2{ // Allowed

}

class D implements I1,I2{ // Allowed

}
```

- 3. Hirerachical

```
class A {

}
class B extends A{

}
class C extends A{

}
```

- 4. Multilevel

```
class A {

}
class B extends A{

}
class C extends B{

}
```

- Hybrid inheritance: Any combination of above types

Super Keyword

- In sub-class, super-class members are referred using "super" keyword.
- used for calling super class constructor
- By default, when sub-class object is created, first super-class constructor (param-less) is executed and then sub-class constructor is executed.
- "super" keyword is used to explicitly call super-class constructor.
- Super class members (non-private) are accessible in sub-class directly or using "this" reference. These members can also be accessed using "super" keyword.
- However, if sub-class method signature is same as super-class signature, it hides/shadows method of the super class i.e. super-class method is not directly visible in sub-class.
- The "super" keyword is mandetory for accessing such hidden members of the super-class.

```
class Employee {
    private int id;
    private String name;
    private double salary;

    public Employee() {
        System.out.println("Employee Ctor");
    }

    public Employee(int id, String name, double salary) {
        System.out.println("Employee Parameterized Ctor");
        this.id = id;
        this.name = name;
        this.salary = salary;
    }

    public void displayEmployee() {
        System.out.println("id = " + id);
        System.out.println("Name = " + name);
        System.out.println("Salary = " + salary);
    }
}

class Manager extends Employee {
    private double bonus;

    public Manager() {
        System.out.println("Manager Ctor");
    }

    public Manager(int id, String name, double salary, double bonus) {
        super(id, name, salary);
        System.out.println("Manager Parameterized Ctor");
        this.bonus = bonus;
    }

    public void displayManager() {
        displayEmployee();
        System.out.println("Bonus = " + bonus);
    }
```

```
}

class Salesman extends Employee {
    private int salesCount;
    private double commission;

    public Salesman() {
        System.out.println("Salesman Ctor");
    }

    public Salesman(int id, String name, double salary, int salesCount, double
commission) {
        super(id, name, salary);
        System.out.println("Salesman Parameterized Ctor");
        this.salesCount = salesCount;
        this.commission = commission;
    }

    public void displaySalesman() {
        displayEmployee();
        System.out.println("SalesCount = " + salesCount);
        System.out.println("Commission = " + commission);
    }

}

public class Program01 {

    public static void main(String[] args) {
        Manager m1 = new Manager();
        Manager m2 = new Manager(1, "Anil", 10000, 5000);

        Salesman s1 = new Salesman();
        Salesman s2 = new Salesman(2, "Mukesh", 20000, 100, 20);
    }
}
```


Method Overriding

- Redefining a super-class method in sub-class with exactly same signature is called as "Method overriding".
- Programmer should override a method in sub-class in one of the following scenarios
    - 1. Super-class has not provided method implementation at all (abstract method).
    - 2. Super-class has provided partial method implementation and sub-class needs additional code. Here sub-class implementation may call super-class method (using super keyword).
    - 3. Sub-class needs different implementation than that of super-class method implementation.

```
class Employee {
    private int id;
    private String name;
    private double salary;

    public void accept() {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter id = ");
        id = sc.nextInt();
        System.out.print("Enter name = ");
        name = sc.next();
        System.out.print("Enter salary = ");
        salary = sc.nextDouble();
    }

    public void display() {
        System.out.println("id = " + id);
        System.out.println("Name = " + name);
        System.out.println("Salary = " + salary);
    }
}

class Manager extends Employee {
    private double bonus;

  // Method Overriding
    public void accept() {
        Scanner sc = new Scanner(System.in);
        super.accept();
        System.out.print("Enter bonus = ");
        bonus = sc.nextDouble();
    }

  // Method Overriding
    public void display() {
        super.display();
        System.out.println("Bonus = " + bonus);
    }
}

class Salesman extends Employee {
    private int salesCount;
    private double commission;

    // Method Overriding
  public void accept() {
        Scanner sc = new Scanner(System.in);
        super.accept();
        System.out.print("Enter sales count = ");
        salesCount = sc.nextInt();
        System.out.print("Enter commission = ");
        commission = sc.nextDouble();
    }
```

```
    // Method Overriding
    public void display() {
        super.display();
        System.out.println("SalesCount = " + salesCount);
        System.out.println("Commission = " + commission);
    }
}

public class Program01 {

    public static void main(String[] args) {
        Employee e = new Employee();
        Manager m = new Manager();
        Salesman s = new Salesman();

        e.accept();
        e.display();

        m.accept();
        m.display();

        s.accept();
        s.display();
    }
}
```